

Lab 8

Edge Impulse Link: <https://studio.edgeimpulse.com/public/1011529/live>

Part I

Idle:

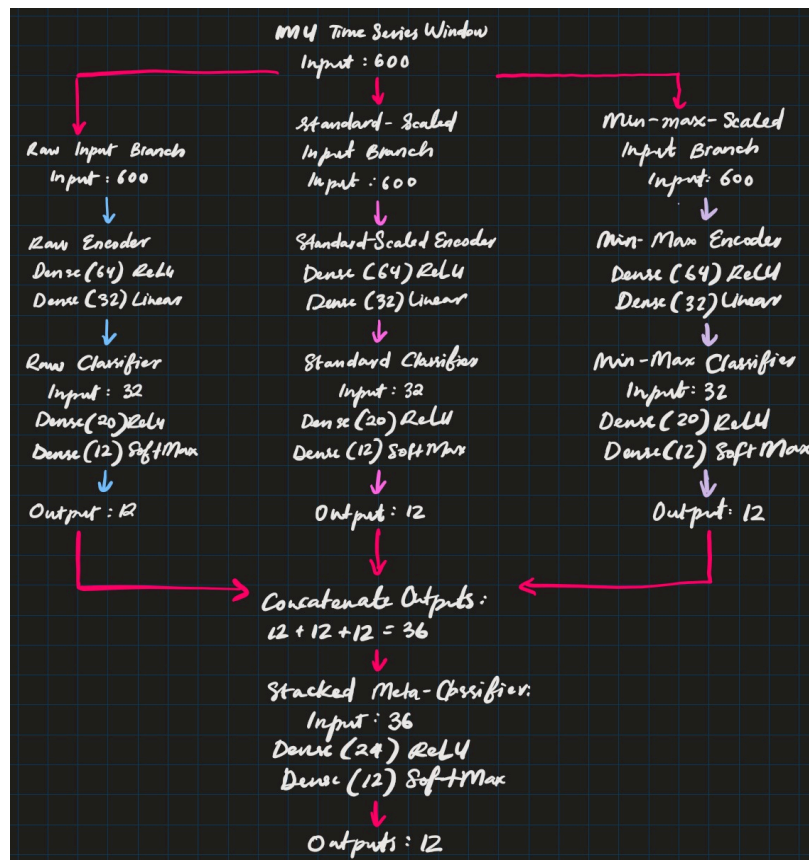
```
Starting inferencing in 2 seconds...
Sampling...
INFO: HW RFFT failed, FFT size not supported. Must be a power of 2 between 32 and 4096, (size was 16)
Timing: DSP 166 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
Idle: 0.996094
Lift: 0.000000
```

Lift:

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 165 ms, inference 570 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
Idle: 0.000000
Lift: 0.996094
```

Part II:

Q1:



Q2:

The notebook compresses the models for TinyML deployment by first applying pruning with TensorFlow Model Optimization's `prune_low_magnitude()` using a polynomial decay schedule that increases sparsity from 10% to 80% over 500 steps. After pruning, the pruning wrappers are stripped so the model becomes a normal sparse Keras model while keeping the zeroed weights, which makes it easier to apply QAT correctly. QAT is then applied by fine-tuning the pruned model with quantization simulation, helping the model adjust before int8 conversion. The notebook also uses mask enforcement during and after QAT because QAT can accidentally make pruned weights nonzero again, and the masks preserve the intended sparsity. Finally, the model is converted to a fully int8 TFLite model using representative data, which calibrates activation ranges for proper scale and zero-point values. Together, pruning and int8 quantization reduce model size, memory use, and computation cost, making deployment more realistic on a resource-limited board like the Arduino Nano 33 BLE Sense.

Q3:

When the Arduino Nano 33 BLE Sense was placed flat on a stable surface, the model did not produce an ideal resting-board prediction. In the first inference window, the final output was:

```
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0039, 0.4883, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.4883, 0.0195, 0.0000
Standard-scaled model scores: 0.0039, 0.0000, 0.9922, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.7812, 0.2187, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0000, 0.3281, 0.0391, 0.0000, 0.0000, 0.1328, 0.0039, 0.0078, 0.0039, 0.4805, 0.0000, 0.0000
-----
Final prediction: Jogging
Class index: 9
Confidence score: 0.4805
-----
```

This is not acceptable for a still board because jogging is a high-motion activity, while the board was stationary. For a resting board, a more reasonable prediction would be a low-motion class such as standing still, sitting, or lying down, depending on how the original dataset labeled resting positions.

The prediction behavior also appeared unstable. Instead of producing one clear resting prediction, the meta-classifier spread its scores across several classes. This suggests that the live Arduino data did not closely match the type of data the model saw during training. As a result, the model was forced to classify unfamiliar input patterns into one of the known training classes, even when none of them were a good match.

Two practical changes that could improve deployment reliability are to collect real Arduino IMU data for the target activities and retrain or fine-tune the model using that data. Another improvement would be to carefully verify that the Arduino sketch applies the same preprocessing as the notebook, including the same feature order, axis orientation, scaling parameters, sampling rate, and unit conversions.

Q4:

Unexpected prediction behavior was observed after deploying the ensemble model on the Arduino Nano 33 BLE Sense. While the board was flat and still, the model predicted Jogging which is not appropriate for a resting condition. This likely happened because the live Arduino sensor data did not match the original training data. Possible causes include differences in sensor placement, axis orientation, accelerometer/gyroscope units, feature scaling, and motion patterns. The raw, standard-scaled, and min-max-scaled branches also produced different class scores, showing that the ensemble was uncertain. Overall, the behavior suggests a mismatch between the training dataset and the real Arduino deployment setup. Reliability could be improved by collecting Arduino-specific training data and making sure the Arduino preprocessing matches the notebook exactly.